



Prueba Técnica de Ingreso Desarrollador

Especificación asistida por IA · 48 horas de reloj más una defensa de 30 minutos

TBTB Global · Departamento de Tecnología · Septiembre de 2026

Antes de empezar: qué evaluamos y qué no

En TBTB Global el desarrollo no arranca escribiendo código. Arranca con un PRD, un paquete de contexto e intención, un plan que se cierra antes de tocar el editor y una bitácora que deja trazable cada decisión. Trabajamos para la industria farmacéutica, donde un registro no se puede sobrescribir sin dejar rastro y donde un criterio de aceptación mal entendido cuesta más que un bug. El código lo escribimos con asistentes de IA todos los días.

Por eso esta prueba no te pide un CRUD, cinco endpoints y una tabla con filtros: hoy eso lo genera un agente en veinte minutos y no nos diría nada sobre ti. Lo que necesitamos saber es otra cosa:

1. **Si lees una especificación de forma crítica:** si detectas los vacíos, las contradicciones y los supuestos peligrosos antes de construir sobre ellos.
2. **Si sabes acotar:** si defines un alcance, lo defiendes y dices con claridad qué dejas fuera y por qué.
3. **Si diriges a la IA o la IA te dirige:** si el resultado es una decisión tuya que la herramienta ejecutó, o una salida que aceptaste sin revisar.
4. **Si dejas evidencia:** si alguien puede seguir tu rastro desde un criterio de aceptación hasta la prueba que lo demuestra.
5. **Si sostienes tus decisiones:** si puedes explicar cualquier línea de lo que entregas y reaccionar cuando el requerimiento cambia.

No evaluamos la cantidad de funcionalidad entregada, ni la belleza de la interfaz, ni si usaste el framework de moda. Una entrega deliberadamente pequeña, bien acotada y bien argumentada puntúa por encima de una entrega completa que no puedes explicar.

Reglas sobre el uso de inteligencia artificial

El uso de asistentes de IA está permitido y esperado. Es la forma en que trabaja el equipo al que aspiras a entrar. Las reglas son tres:

1. **Declara las herramientas:** indica en la bitácora qué asistentes usaste (Claude Code, Copilot, Cursor, ChatGPT u otros) y en qué partes.
2. **Registra las decisiones, no las conversaciones:** no queremos el historial completo del chat. Queremos saber qué te propuso la herramienta, qué aceptaste, qué rechazaste y por qué.
3. **Todo lo que entregas es tuyo:** en la defensa te vamos a pedir que expliques fragmentos concretos, que justifiques un índice, un tipo de dato o un manejo de error. "Eso lo generó la IA" no es una respuesta válida. Si no puedes sostener una línea, bórrala antes de entregar.

Declarar cero uso de IA no suma puntos

Una entrega que declara no haber usado ninguna herramienta y muestra señales evidentes de lo contrario es motivo de descarte. Una que declara su uso con criterio es exactamente lo que buscamos.

Lo que recibes

Junto con este documento recibes el paquete de contexto del ejercicio:

1. **Extracto del PRD (anexo de este documento):** un recorte de un documento de requerimientos de un caso real anonimizado. Está incompleto a propósito, tal como llegan los documentos reales.
2. **Criterios de aceptación:** incluidos dentro del mismo anexo, numerados como [CA-1](#) a [CA-6](#).
3. **Stack y restricciones:** están en este mismo documento, en la Parte III. Son los del equipo en producción y aplican por igual a todos los candidatos.

PARTE I · PESO 30 %

Lectura crítica de la especificación

Antes de escribir una línea de código, lee el extracto del PRD que va al final de este documento y produce un documento de hallazgos. Un hallazgo es cualquier punto del documento que no se puede implementar tal como está escrito: un dato que falta, una regla que se contradice con otra, un caso que nadie definió, un límite que no existe o un riesgo que el documento no ve.

Entrega un archivo `01-hallazgos.md` con una tabla de esta forma:

Campo	Qué esperamos
Referencia	Sección o criterio del PRD donde aparece el hallazgo
Tipo	Vacio, contradicción, ambigüedad, riesgo o límite no definido
Descripción	Qué está mal o qué falta, en una o dos frases
Impacto	Qué pasa si se construye sin resolverlo
Pregunta al PO	La pregunta exacta que harías para desbloquearlo
Supuesto	Con qué asumes que sigues si nadie te responde antes del cierre de la entrega

No hay un número correcto de hallazgos. Hay hallazgos que importan y hallazgos decorativos: preferimos cuatro que bloquean la construcción antes que quince cosméticos. Y ten en cuenta el sector: parte de lo que falta en ese documento falta porque quien lo escribió no pensó en un entorno regulado.

PARTE II · PESO 20 %

Plan y cierre de alcance

Con tus hallazgos y supuestos sobre la mesa, define el plan. Entrega un archivo `02-plan.md` con:

- 1. Alcance cerrado:** qué criterios de aceptación vas a cubrir en esta entrega.
- 2. Fuera de alcance:** qué dejas fuera de forma explícita y con qué justificación. Esta sección pesa igual que la anterior.
- 3. Modelo de datos:** las entidades, sus campos, tipos, restricciones y relaciones, con una nota sobre cómo tratas la corrección de un registro ya guardado.
- 4. Contrato de la interfaz:** los endpoints o las pantallas que vas a construir, con su entrada, su salida y sus errores.
- 5. Secuencia de trabajo:** las tareas en el orden en que las vas a ejecutar y el tiempo que le asignas a cada una.
- 6. Riesgos:** qué puede salirte mal en las próximas horas y cómo lo mitigas.
- 7. Extensión móvil:** media página, sin código. La misma funcionalidad la usa un gestor en campo desde el teléfono, muchas veces sin señal. Explica qué guardarías en el dispositivo, cuándo sincronizas, y qué haces si el mismo registro se modificó en el servidor mientras el teléfono estaba desconectado. Se evalúa el criterio, no la implementación.

Requisito de secuencia, verificable en el historial

El commit de `01-hallazgos.md` y `02-plan.md` debe ser anterior al primer commit de código. Vamos a mirar el historial del repositorio. Un plan escrito después de construir no cuenta como plan, y esta es la parte del ejercicio que más se parece a cómo trabaja el equipo: aquí el plan se cierra antes, y cambiarlo después exige decir por qué.

PARTE III · PESO 20 %

Una funcionalidad completa de punta a punta

Implementa el alcance que cerraste atravesando las tres capas: la tabla en la base de datos, el servicio o API que la expone y la pantalla desde la que se usa. Una sola funcionalidad que recorra todo el camino y quede andando vale más que tres funcionalidades a medias en capas distintas.

Tope de alcance: dos o tres criterios de aceptación

Cubre como mínimo dos de los seis criterios del anexo y como máximo tres. Pasar de tres no suma puntos y sí resta, porque el ejercicio está diseñado para que sobre tiempo de pensar y falte tiempo de teclear. Elegir cuáles dos o tres, y justificarlo en el plan, es parte de lo que evaluamos.

Requisitos mínimos:

1. **Persistencia versionada:** el esquema se crea con scripts o migraciones versionadas, no a mano desde una herramienta gráfica. Van en una carpeta `scripts/` en la raíz del repositorio.
2. **Separación de responsabilidades:** la lógica de negocio no vive en el controlador ni en el componente de interfaz.
3. **Pruebas atadas a los criterios:** al menos una prueba automatizada por cada criterio de aceptación que declaraste en alcance. El nombre de la prueba debe permitir identificar el criterio que verifica.
4. **Manejo de error visible:** al menos un caso de error tratado de punta a punta, desde la validación hasta lo que ve el usuario.
5. **Datos de prueba:** un script que cargue datos suficientes para que podamos ver la funcionalidad andando sin inventarlos nosotros.

Stack

Es el que usa el equipo en producción y aplica a todos por igual. Si algo te resulta inviable en el tiempo disponible, no lo fuerces: dilo en el plan y propón la alternativa, que se evalúa mejor.

1. **Base de datos:** SQL Server. El esquema se crea con scripts `.sql` versionados y numerados, o con migraciones de Entity Framework Core incluidas en el repositorio.
2. **Servicio:** .NET 8 con ASP.NET Core Web API. Capas separadas entre controlador, servicio y acceso a datos, con objetos de transferencia propios en la frontera del API.
3. **Interfaz:** Angular 17 o superior, con componentes tipados y consumo del API por servicio inyectado, no por llamada directa desde el componente. Si prefieres construirla con Ionic sobre Angular, sirve igual y no cambia la calificación.
4. **Pruebas:** xUnit o NUnit para la capa de servicio. Las pruebas de interfaz son opcionales y suman.
5. **Consulta con criterio:** incluye al menos una consulta que resuelva un requerimiento del PRD combinando tablas, y justifica en la bitácora por qué la escribiste así y qué índice necesitaría con volumen real.

PARTE IV · PESO 15 %

Trazabilidad y bitácora

Entrega un archivo `03-bitacora.md` con dos secciones.

Matriz de trazabilidad

Una fila por criterio de aceptación en alcance, con cuatro columnas: el criterio, el commit o los archivos que lo implementan, la prueba que lo verifica y el estado (cubierto, parcial o fuera de alcance). Un criterio declarado en alcance sin prueba que lo respalde cuenta como no cubierto.

Registro de decisiones y uso de IA

Una lista cronológica corta, de una línea por entrada, con las decisiones que tomaste durante el ejercicio. Para cada una: qué decidiste, por qué, y si la propuesta salió de ti o de un asistente. Incluye al menos dos entradas donde hayas rechazado o corregido algo que la herramienta te propuso, explicando el motivo. Si no rechazaste nada en cuatro horas de trabajo, revisa con más cuidado lo que aceptaste.

PARTE V · PESO 15 %

Defensa de 30 minutos

Una vez recibida tu entrega, agendamos una sesión de treinta minutos con dos líderes técnicos de TBTB. No necesitas preparar presentación. La sesión tiene tres bloques:

- 1. Recorrido de tus hallazgos (10 minutos):** nos cuentas qué encontraste en el PRD y qué decidiste hacer con cada cosa.
- 2. Lectura de tu código (10 minutos):** abrimos tu repositorio y te pedimos que expliques fragmentos que elegimos nosotros.
- 3. Cambio en caliente (10 minutos):** te entregamos un cambio de requerimiento que no está en el PRD y nos dices cómo impacta tu modelo de datos, tu contrato, tus pruebas y tu plan. Si quieres implementarlo en vivo con tu asistente de IA compartiendo pantalla, mejor; no es obligatorio.

Entrega

Tiempo

Tienes 48 horas de reloj desde el correo con el que recibes este documento. El esfuerzo estimado del ejercicio es de cuatro a seis horas, y el tope de dos o tres criterios de aceptación está puesto justamente para que quepa ahí sin que dejes de atender tus compromisos actuales. Si terminas antes, con gusto lo tomamos en cuenta.

Si el tiempo no te alcanza, no entregues de más: recorta el alcance en el plan y documenta el recorte. Gestionar el tiempo declarando lo que no vas a hacer es parte de lo que se evalúa.

Repositorio

Sube el ejercicio a un repositorio en la plataforma que prefieras (GitHub, GitLab, Bitbucket) y envíanos la URL. Estructura esperada:

```
raiz/
├── 01-hallazgos.md
├── 02-plan.md
├── 03-bitacora.md
├── README.md           paso a paso para clonar, configurar y levantar
├── scripts/            migraciones y datos de prueba
├── api/                servicio .NET
└── web/                cliente Angular
```

Buenas prácticas que revisamos

- 1. Historial de commits legible:** mensajes que digan qué cambió y por qué, en incrementos que se puedan leer. El historial es parte de la entrega.
- 2. README que funcione:** vamos a seguir tus instrucciones al pie de la letra en una máquina limpia. Si no arranca, no lo adivinamos.
- 3. Sin artefactos de caché:** `node_modules`, `bin`, `obj` y equivalentes fuera del repositorio, con su `.gitignore`.
- 4. Sin secretos en el código:** cadenas de conexión y credenciales por configuración o variables de entorno, con un archivo de ejemplo.
- 5. Formato consistente:** indentación uniforme y, de ser posible, un linter configurado.

Cómo se califica

Bloque	Peso	Qué mueve la nota hacia arriba
Lectura crítica de la especificación	30 %	Hallazgos que bloquean de verdad, preguntas bien formuladas y supuestos con impacto declarado
Plan y cierre de alcance	20 %	Alcance defendible, exclusiones justificadas y plan anterior al código en el historial
Funcionalidad de punta a punta	20 %	Funciona de punta a punta, con separación de responsabilidades y pruebas atadas a criterios
Trazabilidad y bitácora	15 %	Cada criterio rastreable hasta su prueba, y decisiones sobre IA con criterio propio
Defensa	15 %	Explica su propio código y razona el cambio de requerimiento sin improvisar

Se aprueba la prueba con 70 puntos sobre 100, con un mínimo de 15 sobre 30 en el bloque de lectura crítica. Un candidato que no puede explicar su propia entrega en la defensa no aprueba, independientemente de la puntuación de los demás bloques.

Una nota sobre el sentido del ejercicio

Esta prueba es también una muestra del trabajo real. Si el ejercicio te resulta interesante, probablemente el puesto también. Si te resulta burocrático, es información valiosa para los dos.

Te deseamos el mayor de los éxitos.

ANEXO

Extracto del PRD — caso anonimizado

Lo que sigue es un recorte de un documento de requerimientos de un programa de acompañamiento a pacientes, adaptado y anonimizado. Se entrega tal cual: no lo tomes como una especificación correcta.

Contexto del programa

El programa acompaña a pacientes que inician un tratamiento crónico. Un equipo de gestores contacta a cada paciente según un calendario de seguimiento y registra el resultado de cada contacto. El programa opera en Colombia, Perú y Ecuador. Los reportes de adherencia se entregan mensualmente al laboratorio que patrocina el programa.

Alcance funcional solicitado

- 1. Registro de pacientes:** el sistema debe permitir registrar un paciente con su nombre, documento de identidad, teléfono, correo electrónico, ciudad y fecha de inicio de tratamiento. El teléfono es obligatorio porque el contacto principal es telefónico.
- 2. Autorregistro:** el paciente también puede registrarse por sí mismo desde un enlace, indicando su nombre y su correo electrónico.
- 3. Registro de contactos:** el gestor registra cada contacto con el paciente indicando la fecha, el canal utilizado (llamada, WhatsApp o correo) y el resultado del contacto.
- 4. Corrección de registros:** si el gestor se equivocó al registrar un contacto, debe poder corregir el registro para que el reporte salga bien.
- 5. Consulta de contactos del mes:** la coordinadora del programa debe poder ver todos los contactos del mes en curso y filtrarlos por gestor y por ciudad.
- 6. Reporte de adherencia:** el sistema calcula el porcentaje de pacientes contactados dentro del plazo previsto sobre el total de pacientes activos del programa.

Criterios de aceptación

Id	Criterio
CA-1	Dado un paciente nuevo, cuando el gestor completa el formulario de registro, entonces el paciente queda disponible para agendar contactos.
CA-2	Dado un paciente registrado, cuando el gestor registra un contacto, entonces el contacto queda asociado al paciente con su fecha, canal y resultado.
CA-3	Dado un contacto registrado con error, cuando el gestor lo corrige, entonces el reporte refleja la información corregida.
CA-4	Dada la vista de contactos del mes, cuando la coordinadora filtra por gestor y ciudad, entonces solo se muestran los contactos que cumplen ambos filtros.
CA-5	Dado un paciente que no contesta tres veces consecutivas, cuando se genera el reporte mensual, entonces el paciente aparece marcado como ilocalizable.
CA-6	Dado el cierre de mes, cuando la coordinadora genera el reporte de adherencia, entonces el porcentaje se calcula sobre los pacientes activos al último día del mes.

Notas del documento original

La versión definitiva del calendario de seguimiento está pendiente de confirmación con el área médica. El diseño de las pantallas se resolverá con el equipo de diseño más adelante. El programa arranca con alrededor de cuatrocientos pacientes.

Recordatorio

Este extracto es tu insumo, no tu guion. Tu primer entregable es decirnos qué le falta y qué está mal antes de construir sobre él.